Efficient Scheduling for Batched AI Jobs: Minimising Makespan through Model Grouping, Johnson's Rule, and Checkpoint Prefetching

Jingsheng

4th Year Project Report
Computer Science
School of Informatics
University of Edinburgh

2026

Abstract

Large Language Model (LLM) batch workloads—model benchmarking, dataset labelling, and synthetic data generation—submit hundreds of tasks that share common model weights. Existing serverless LLM schedulers treat tasks as independent, causing excessive model switching that dominates total execution time.

This dissertation formalises the batch scheduling problem as a two-machine flow-shop (I/O and GPU computation) and introduces three composable optimisations: (1) **model grouping** reduces model switches from $O(n)$ to $O(k)$; (2) **Johnson's Rule ordering** sequences model groups to maximise I/O-computation overlap; and (3) **checkpoint prefetching** preloads the weights of the next model into CPU pinned memory while the current group executes.

Evaluation on NVIDIA RTX A5000 GPUs shows that model grouping reduces makespan by 99% over FIFO (78,900s to 1,107s for a 500-task batch with five models). Checkpoint prefetching hides 68.7% of model loading latency. Johnson's Rule provides 7.2–22.0% additional makespan reduction over naive orderings. The shared-aware strategy outperforms FIFO by 49–237$\times$ across batch sizes of 100–10,000 tasks and model sizes of 0.6B–32B, with advantages growing as batch size increases. Prefetching remains effective across storage tiers including 10 GbE network storage (78.9% of I/O latency hidden), demonstrating the viability of remote storage in production deployments.

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(*Jingsheng*)

Acknowledgements

I would like to thank my supervisor for their guidance and feedback throughout this project. I am grateful to the ServerlessLLM team for providing the open-source framework that made this work possible, and to the School of Informatics for providing access to the GPU cluster used for experimental evaluation.

Table of Contents

Chapter 1

Introduction

Large Language Model (LLM) batch workloads, including model benchmarking, dataset processing, and synthetic data generation, submit thousands of tasks together for offline processing. Unlike online serving, batch tasks are not independent: they share substantial computational structure, including identical model weights and common prompt prefixes, that can be exploited to reduce total execution time.

1.1 Motivation

Consider a model benchmarking scenario: evaluating two LLMs on 10 questions each, totalling 20 tasks. A naive scheduler executes tasks in submission order (ABABAB...), repeatedly loading and evicting model weights. Each model switch takes roughly 100 seconds while each inference requires under a second, so the system spends over 96% of its time on I/O. Grouping tasks by model (AAAA...BBBB) reduces switches from 10 to 1, cutting execution time by 87% (Figure 1.1).

Reducing model switches from $O(n)$ to $O(k)$ through grouping provides an order-of-magnitude improvement, but does not eliminate the remaining $O(k)$ model transitions. Checkpoint prefetching hides this residual latency by loading the next model's weights into CPU pinned memory while the current group executes on GPU, exploiting the independence of I/O and GPU computation.

Prefetching effectiveness depends on the *order* in which groups execute: a compute-heavy group provides a large overlap window, while a short group provides little. This structure, where each model group passes through two sequential stages (I/O loading then GPU computation) with the objective of minimising total completion time, is precisely the classical *two-machine flow-shop* problem Johnson (1954). Johnson's Rule provides the provably optimal group ordering by placing compute-heavy groups first, maximising the overlap window for prefetching.

The three components compose into a complete pipeline: grouping reduces switches from $O(n)$ to $O(k)$, prefetching hides I/O behind computation, and Johnson's Rule maximises overlap. Existing serverless LLM schedulers treat batch tasks as independent, causing excessive model switching and no I/O-computation overlap.

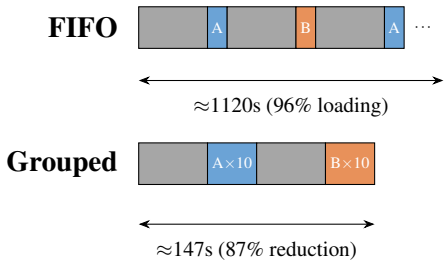

Figure 1.1: FIFO vs Model-Grouped scheduling for 20 tasks with 2 models. FIFO incurs 10 switches (each ∼108s); grouping requires only 1 switch.

1.2 Problem Definition

A batch is submitted as a JSONL file where each line specifies one task: the target model, the prompt, and generation parameters. The example below shows a small multi-model batch typical of model benchmarking.

```
{"custom_id":"t1","method":"POST","url":"/v1/chat/completions
   ",
 "body":{"model":"Qwen/Qwen3-0.6B",
         "messages":[{"role":"user","content":"Summarise the
            paper."}],
         "max_tokens":256}}
{"custom_id":"t2","method":"POST","url":"/v1/chat/completions
   ",
 "body":{"model":"Qwen/Qwen3-8B",
         "messages":[{"role":"user","content":"Summarise the
            paper."}],
         "max_tokens":256}}
{"custom_id":"t3","method":"POST","url":"/v1/chat/completions
   ",
 "body":{"model":"Qwen/Qwen3-0.6B",
         "messages":[{"role":"user","content":"Translate to
            French."}],
         "max_tokens":128}}
```

Tasks `t1` and `t3` share the same model and can execute consecutively without reloading; `t2` requires a model switch. The API follows the OpenAI Batch API specification (OpenAI, 2024): clients submit a file via `POST /v1/files`, create a batch via `POST /v1/batches`, and poll via `GET /v1/batches/{id}`. Results are written to an output JSONL file keyed by `custom_id`.

Input:

- A batch of n tasks $B = \{t_1, t_2, \ldots, t_n\}$
- Each task $t_i = (m_i, p_i, \theta_i)$ where m_i is the model, p_i is the prompt, and θ_i are generation parameters
- A cluster of k GPU workers $W = \{w_1, w_2, \ldots, w_k\}$

- Each worker can load at most one model at a time, with loading latency $L_{load}(m)$
- Task inference latency is $L_{infer}(t_i)$

Sharing Relationships:

- **Model Sharing**: If $m_i = m_j$, tasks t_i and t_j can share loaded model weights, avoiding redundant loading. This is the primary sharing relationship exploited in this work.
- **Prefix Sharing**: If $\text{prefix}(p_i) = \text{prefix}(p_j)$, tasks can benefit from inference engine cache reuse. This is a secondary sharing relationship identified but not exploited in this work (see Chapter 7).

Optimisation Objective:

$$\text{Minimise } C_{max} = \max_{i \in \{1,\ldots,n\}} \text{completion\_time}(t_i)$$

That is, minimise the makespan: the time at which the last task completes.

Constraints:

- Each worker loads at most one model at a time (single-model constraint)
- Model switching incurs L_{load} overhead (cold start penalty)
- Worker count k can be adjusted via auto-scaling

1.3 Research Questions

Three research questions are addressed:

RQ1: How can shared structure (shared models) in a batch be discovered and exploited to reduce makespan? Specifically, how does model grouping compare to naive FIFO scheduling?

RQ2: Can the batch scheduling problem be modelled as a two-machine flow-shop (I/O and compute), and does Johnson's Rule provide an effective ordering of model groups to maximise I/O-computation overlap via checkpoint prefetching?

RQ3: How do different scheduling strategies (FIFO, Model Grouping, Shared-Aware with prefetching) compare across varying batch sizes, model sizes, and storage tiers (local SSD, RAID, network storage)?

1.4 Contributions

Five contributions are made:

1. **Two-Machine Flow-Shop Formulation**: The batch scheduling problem is modelled as a classical two-machine flow-shop where Machine A is I/O (loading model weights) and Machine B is GPU computation, enabling Johnson's Rule for provably optimal model group ordering.

2. **Shared-Aware Scheduling Algorithm**: An algorithm that discovers model sharing relationships within a batch, groups tasks by model to reduce switches from $O(n)$ to $O(k)$, and applies Johnson's Rule to order groups for maximal I/O-computation overlap.
3. **Checkpoint Prefetching Mechanism**: A prefetching policy using the pinned memory pool of ServerlessLLM to preload the next model's weights while the current group executes, hiding I/O latency through pipelining.
4. **Network-Aware Prefetching Extension**: Analysis and evaluation of prefetching across storage tiers including RAID-0, 10 GbE NFS, and 100 GbE NFS, demonstrating that the prefetch architecture generalises to network-attached storage.
5. **OpenAI-Compatible Batch API**: An OpenAI-compatible batch REST API extended for multi-model batches, enabling clients to submit heterogeneous task lists with a single request and receive results asynchronously.

The project contributed approximately 3,500 lines of new Python code to the existing ServerlessLLM framework ($\sim$15K LoC).

1.5 Scope and Boundaries

This work focuses strictly on cluster-level scheduling. The inference engine is neither modified nor optimised; KV cache management, prefill-decode separation, continuous batching internals, and GPU kernel optimisations are out of scope. The scheduler decides which tasks to execute, in what order, and on which workers; vLLM handles the computation. Scheduler-level optimisations are complementary to engine-level optimisations and require no engine modification.

Chapter 2

Background

2.1 Transformer-Based LLM Inference

LLMs are built on the transformer architecture (Vaswani et al., 2017), generating tokens autoregressively via a compute-bound prefill phase ($O(L^2)$ attention) followed by a memory-bandwidth-bound decode phase (one token per step). Modern LLMs range from 0.5B to 405B parameters (Brown et al., 2020; Touvron et al., 2023); an 8B model requires $\sim$15 GB in FP16 and a 70B model $\sim$140 GB. Loading weights from storage takes 80–120 seconds for an 8B model from SSD.

vLLM (Kwon et al., 2023) introduced continuous batching (iteration-level scheduling), inserting new requests at every decode step to maintain near-full GPU utilisation within a single loaded model. It does not address switching between models. Tensor parallelism (Shoeybi et al., 2019) shards weight matrices across multiple GPUs for models exceeding a single GPU's VRAM; this work assumes each model fits within one GPU, and tensor parallelism can be combined with the proposed scheduler when needed.

2.2 KV Cache and Prefix Caching

Inference engines cache key-value tensors to avoid redundant computation. vLLM manages the KV cache in fixed-size blocks via PagedAttention (Kwon et al., 2023), eliminating fragmentation. When requests share a prompt prefix, KV blocks can be computed once and reused (Zheng et al., 2024), though interleaving different prefixes causes eviction. This prefix-level optimisation is complementary to model grouping but is not exploited here (see Chapter 7).

2.3 GPU Memory Hierarchy and Pinned Memory

Loading model weights proceeds in two stages: SSD to CPU RAM via filesystem I/O (2–5 GB/s on NVMe), then CPU RAM to GPU VRAM via PCIe DMA (16–32 GB/s on PCIe Gen4 x16). The CPU-to-GPU step is faster with pinned (page-locked) memory,

allocated via `cudaHostAlloc` or `cudaHostRegister` (NVIDIA, 2024a), which enables direct DMA without intermediate copies. The sllm-store component of ServerlessLLM (Fu et al., 2024) pre-allocates a pinned memory pool at startup; weights are loaded into this pool via multi-threaded I/O and transferred to the GPU via DMA, achieving near-peak PCIe bandwidth.

2.4 Serverless Computing and Cold Starts

Serverless functions suffer from cold starts: the latency of initialising a new instance (Shahrad et al., 2020). For LLM inference these are severe; loading an 8B model takes 80–120 s, some 100–200× the per-request inference latency. ServerlessLLM (Fu et al., 2024) mitigates this through locality-enhanced scheduling that routes inference to nodes with locally cached checkpoints.

2.5 ServerlessLLM Architecture

ServerlessLLM (Fu et al., 2024) is an open-source serverless LLM inference framework designed for low-latency model loading. Its four components are: a **Head Node**, which receives requests via an API gateway, maintains deployment state in SQLite, and dispatches tasks via a Router; **Workers (Pylets)**, GPU nodes each running vLLM for inference with Pylet managing instance lifecycle; **sllm-store**, a C++ storage daemon per worker managing a pinned memory pool and providing gRPC endpoints for model loading (`LoadModelAsync`, `RegisterModel`) with multi-threaded I/O to saturate SSD bandwidth (gRPC Authors, 2024); and an **AutoScaler**, which reactively adjusts vLLM instance count based on queue depth.

The existing scheduler targets online serving via round-robin assignment, with two limitations for batch workloads: tasks are assigned without considering shared models, causing excessive switching, and scaling is reactive rather than proactive, causing cold starts at batch start.

2.6 Storage Hierarchy and Network-Attached Model Repositories

The storage tier determines I/O time a_i in the two-machine flow-shop formulation, directly affecting prefetching effectiveness. Local NVMe SSDs provide 2–5 GB/s; RAID-0 (4×) aggregates to 8–12 GB/s. Remote storage (S3, GCS, NFS, Lustre) provides unlimited capacity and replication at 1.2–12 GB/s. Production systems use local SSD as a cache and remote storage as the source of truth. Scheduling implications are analysed in Section 4.4.2.

2.7 Makespan Minimisation

Makespan minimisation assigns n jobs to m machines to minimise $C_{\max}$ (Pinedo, 2016). The Longest Processing Time (LPT) rule (Graham, 1966) gives a $(4/3 - 1/(3m))$-approximation for identical machines. LLM batch scheduling has sequence-dependent setup times (model loading: $\sim$100 s per switch, zero for same-model jobs), making the general problem NP-hard (Pinedo, 2016).

2.7.1 Johnson's Rule for Two-Machine Flow-Shops

Johnson's Rule (Pinedo, 2016) solves the two-machine flow-shop optimally: given n jobs processed first on Machine A then Machine B, find the permutation minimising makespan. Partition jobs into $S_1 = \{j : a_j \leq b_j\}$ (compute-heavy, sorted by a_j ascending) and $S_2 = \{j : a_j > b_j\}$ (I/O-heavy, sorted by b_j descending); the optimal sequence is S_1 followed by S_2. Compute-heavy jobs execute first because their prolonged Machine B execution conceals the loading of the next job on Machine A; I/O-heavy jobs are placed last since fewer subsequent jobs remain to stall while their loading completes.

Machine A is the I/O stage (loading weights from SSD to CPU pinned memory), Machine B is the compute stage (GPU inference for all tasks in a group), and each job is a model group. This mapping is formalised in Chapter 4.

Chapter 3

Related Work

3.1 LLM Batch Workloads

LLM batch workloads take three common forms: model benchmarking (evaluating candidates on suites such as MMLU (Hendrycks et al., 2021), HumanEval (Chen et al., 2021), and MT-Bench (Zheng et al., 2023), with high prompt overlap but diverse model identities); dataset processing (large-scale labelling, translation, or summarisation with a single model); and synthetic data generation (template-based prompts exhibiting both model and prefix overlap).

The OpenAI Batch API (OpenAI, 2024) supports up to 50,000 tasks per batch with 50% cost savings; AWS Bedrock Batch (Amazon Web Services, 2024) provides similar functionality. Both are closed-source, single-model only, and do not expose scheduling policies.

3.2 ML Serving Systems

Clipper (Crankshaw et al., 2017) provides adaptive batching for online latency SLOs. INFaaS (Romero et al., 2021) automates model variant selection, treating switching as a configuration change. Clockwork (Gujarati et al., 2020) achieves predictable DNN serving through profiling, but its deterministic timing assumption does not extend to autoregressive LLMs. AlpaServe (Li et al., 2023) uses statistical multiplexing with model parallelism but does not reorder tasks across models. vLLM (Kwon et al., 2023) combines PagedAttention, continuous batching, and tensor parallelism for single-model serving; multi-model batches still require sequential loads between groups. Orca (Yu et al., 2022) introduced iteration-level scheduling within a single model. None of these systems address cross-model task reordering for makespan minimisation.

3.3 Data-Locality and Model-Aware Scheduling

MapReduce (Dean and Ghemawat, 2008) and Spark (Zaharia et al., 2010) schedule tasks on nodes holding the input data; for LLM inference the analogue is routing to

nodes holding model weights. Borg (Verma et al., 2015) and Kubernetes (The Kubernetes Authors, 2024) provide affinity-based cluster scheduling without model-specific optimisations. Database multi-query optimisers identify common subexpressions across queries (Sellis, 1988), inspiring the present approach of discovering shared structure before optimising execution order. ServerlessLLM (Fu et al., 2024) provides locality-enhanced scheduling with local SSD caching but does not reorder tasks by model or prefix.

3.4 Prefetching and Pipelining

OS-level prefetching uses sequential access patterns (Patterson and Hennessy, 2013). DeepSpeed-Inference (Aminabadi et al., 2022) prefetches transformer layers during inference, overlapping CPU-to-GPU transfer with computation; FlexGen (Sheng et al., 2023) pipelines model offloading across CPU, GPU, and disk for memory-constrained inference; CheckFreq (Mohan et al., 2021) analyses checkpoint I/O costs during training. No existing system performs checkpoint prefetching at the cluster scheduler level. This work introduces task-group-level prefetching, where the scheduler predicts the next model from the execution plan and begins loading it before the current group completes.

3.5 Summary and Research Gap

Table 3.1: Comparison of scheduling approaches for LLM inference

System	Batch Support	Model Grouping	Prefix Sorting	Prefetch	Open Source
Orca (Yu et al., 2022)	×	×	×	×	×
Clipper (Crankshaw et al., 2017)	×	×	×	×	✓
AlpaServe (Li et al., 2023)	×	Placement	×	×	✓
ServerlessLLM (Fu et al., 2024)	×	Locality	×	×	✓
OpenAI Batch (OpenAI, 2024)	✓	N/A*	?	?	×
This work	✓	✓	✓	✓	✓

*OpenAI Batch API is single-model only, so model grouping is not applicable.

Existing systems either focus on online serving or provide batch APIs without shared-structure-aware scheduling. This work fills that gap with a cluster-level scheduler that discovers model sharing, applies Johnson's Rule for optimal group ordering, and uses checkpoint prefetching to hide loading latency.

Chapter 4

Design and Implementation

The shared-aware scheduling system has four components: shared-structure analysis discovering model sharing; a two-machine flow-shop formulation with Johnson's Rule for optimal group ordering; checkpoint prefetching to hide model loading latency; and multi-GPU load balancing.

4.1 System Architecture Overview

The system extends the ServerlessLLM head node with three new components: the BatchScheduler (model grouping and Johnson's Rule ordering), the StorageManager (checkpoint prefetching into the pinned memory pool), and a batch-aware AutoScaler integration, all operating at cluster scheduler level without inference engine modifications. A SQLite database (`state.db`) stores batch jobs, task status, and deployment configuration; the Router reads healthy worker endpoints from it. Data flow: (1) the user submits a batch via the API Gateway; (2) the BatchScheduler groups tasks, applies Johnson's Rule, persists job state, and feeds ordered tasks to the Router; (3) the Router dispatches tasks to vLLM workers; (4) the StorageManager prefetches the next model's weights while the current group executes; (5) the AutoScaler adjusts instance count.

Figure 4.1 illustrates the system architecture.

4.2 Problem Analysis: Shared Structure in Batches

4.2.1 Model Sharing Analysis

Batch workloads exhibit substantial model sharing: in benchmarking, multiple tasks use the same model with different prompts; in dataset processing, all tasks share one model. With arbitrary ordering across k distinct models, the system incurs $O(n)$ switches in the worst case, each requiring eviction and reload (80–120 seconds for an 8B model). Grouping reduces switches to $O(k)$; for 20 interleaved tasks across two models, this eliminates $9 \times 108 = 972$ seconds of overhead. Since L_{load} is 100–200$\times$ larger than L_{infer}, minimising switches has first-order impact on makespan.

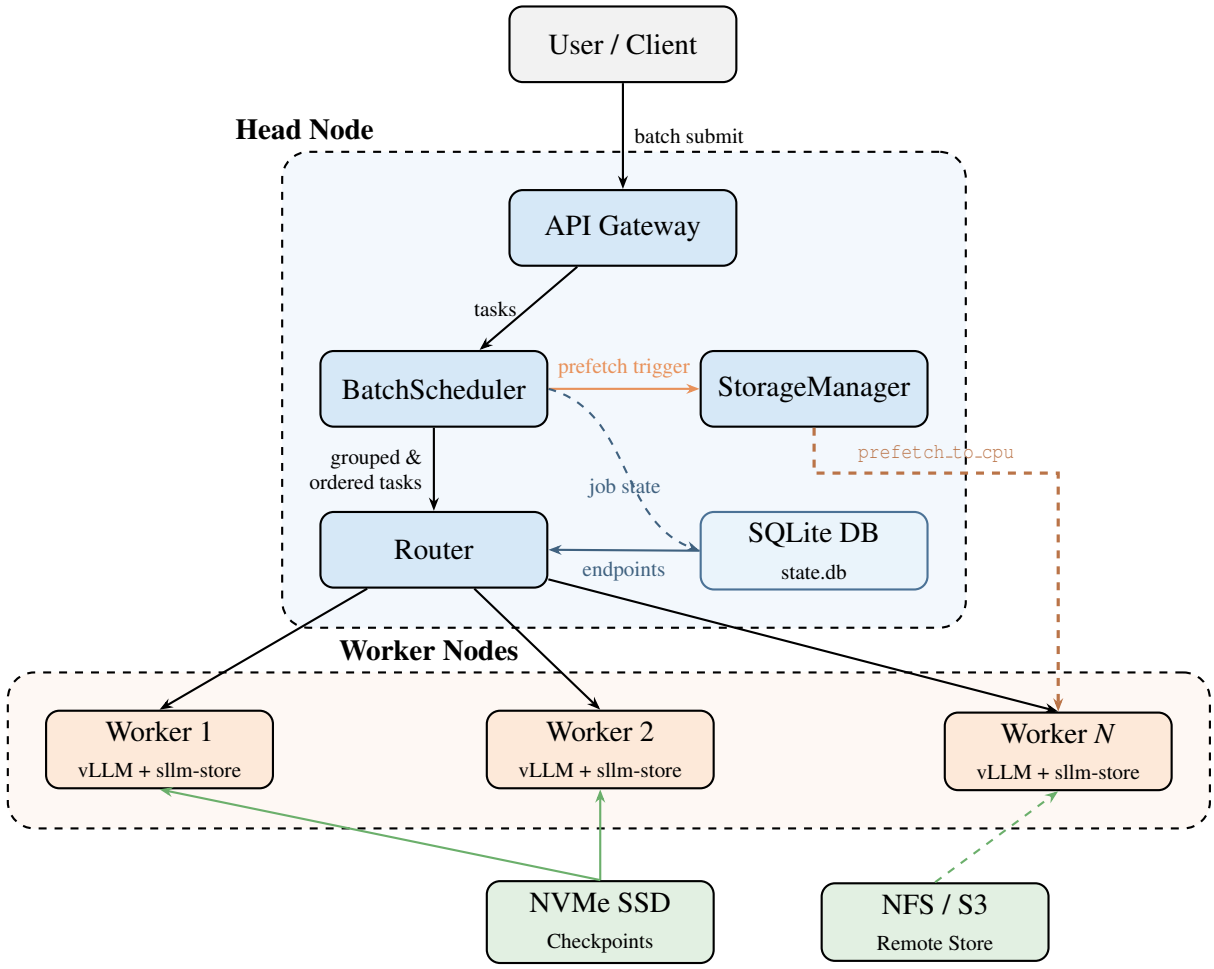

Figure 4.1: System architecture. The BatchScheduler groups tasks by model, applies Johnson's Rule ordering, and persists job state to a SQLite database. The Router queries the same database for healthy worker endpoints. The StorageManager triggers asynchronous checkpoint prefetching (`prefetch_to_cpu`) to worker nodes ahead of execution. Workers run vLLM with sllm-store for pinned-memory checkpoint management.

4.2.2 Prefix Sharing Analysis

Batch tasks often share prompt prefixes (RAG contexts, few-shot examples, system prompts). Prefix caching reduces prefill time from $\sim$2 s to $\sim$0.05 s, but model switching costs 80–120 s, making model grouping the first-order optimisation. Prefix-aware ordering within groups is left as future work (Chapter 7).

4.3 Shared-Aware Scheduling Algorithm

4.3.1 Phase 1: Model Grouping

Given a batch $B = \{t_1, t_2, \ldots, t_n\}$, tasks are grouped by model to minimise model switches:

$groups \leftarrow \{\}$
for $t_i \in B$ **do**
  $m \leftarrow t_i.model$
  if $m \notin groups$ **then**
    $groups[m] \leftarrow []$
  end if
  $groups[m]$.append(t_i)
end for

This produces k model groups $\{G_1, \ldots, G_k\}$ where each G_i contains all tasks for one model, reducing switches from $O(n)$ to $O(k)$.

4.3.2 Phase 2: Johnson's Rule Ordering

Model groups map naturally to a two-machine flow-shop: each group is first loaded from storage (Machine A: I/O) then executed on GPU (Machine B: compute). With checkpoint prefetching, loading the next group overlaps with executing the current one. Johnson's Rule determines the group ordering that minimises makespan.

For each model group G_i, two times are estimated:

- a_i (I/O time): the time to load model m_i's checkpoint from SSD to CPU pinned memory, estimated as $a_i = \text{checkpoint\_size}(m_i)/\text{NVMe\_bandwidth}$.
- b_i (compute time): the time to execute all tasks in G_i on GPU, estimated as $b_i = |G_i| \times \tau_{base} \times (\text{model\_size}(m_i)/\text{base\_model\_size})$, where τ_{base} is the base per-task inference time.

Checkpoint size is determined by scanning the model directory for weight files (`.safetensors`, `.bin`, `.pt`), cached after the initial scan. Figure 4.2 illustrates this mapping.

Johnson's Rule: partition into $S_1 = \{G_i : a_i \leq b_i\}$ (compute-heavy, sorted by a_i ascending) and $S_2 = \{G_i : a_i > b_i\}$ (I/O-heavy, sorted by b_i descending); optimal sequence is $S_1 + S_2$.

$S_1 \leftarrow \{G_i : a_i \leq b_i\}$, sorted by a_i ascending
$S_2 \leftarrow \{G_i : a_i > b_i\}$, sorted by b_i descending

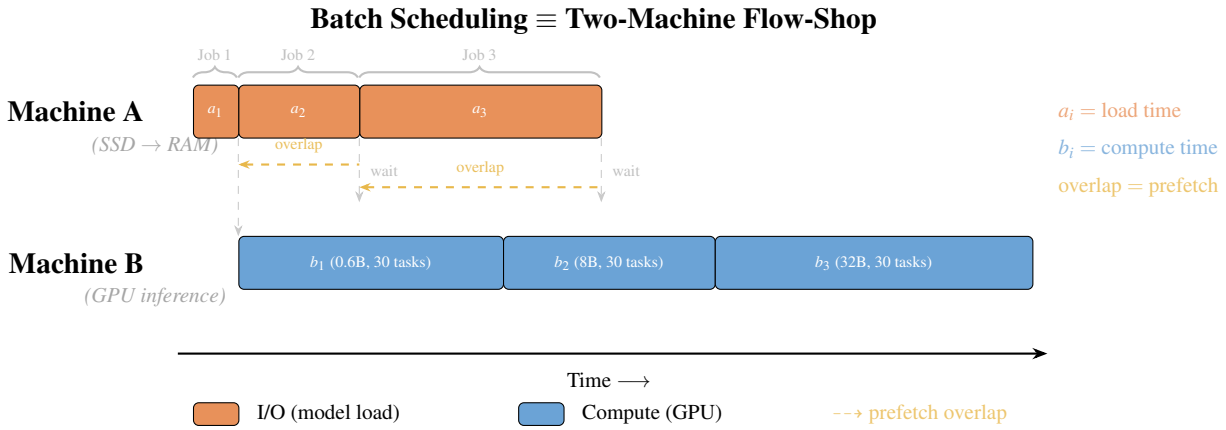

Figure 4.2: Mapping from batch scheduling to two-machine flow-shop. Each model group G_i becomes a "job" with I/O time a_i (loading weights from SSD to pinned memory, Machine A) and compute time b_i (GPU inference for all tasks, Machine B). Checkpoint prefetching enables the I/O of job $i+1$ to overlap with the compute of job i. Johnson's Rule finds the permutation π that minimises the total makespan by maximising these overlaps.

$$\pi \leftarrow S_1 + S_2$$

Why Johnson's Rule outperforms naive ordering. Consider Group X ($a = 10$s, $b = 90$s, compute-heavy) and Group Y ($a = 90$s, $b = 10$s, I/O-heavy). Under X→Y, Y's 90s load fully overlaps with X's 90s compute, giving ≈110s total. Under Y→X, only 10s of X's load overlaps with Y's compute, leaving an 80s stall for ≈190s total. Johnson's Rule correctly places X first.

4.3.3 Phase 3: Worker Allocation

For multi-GPU deployments, model groups are sorted by total estimated time ($a_i + b_i$) descending and greedily assigned to the GPU with the least load; if the slowest GPU's load exceeds the fastest by more than 15%, tasks migrate until balance is achieved. Each GPU's queue is then independently ordered using Johnson's Rule to maximise local I/O-computation overlap.

Models exceeding a single GPU's memory use tensor parallelism (TP). The scheduler handles TP models through capacity-aware lane packing: each TP group occupies a dedicated execution lane of *tp* GPU slots. When budget allows, multiple TP groups run in parallel on non-overlapping GPU subsets; otherwise, groups sharing the same TP size execute sequentially in one lane. Groups are packed in first-fit decreasing order (largest TP first), reserving at least one GPU for single-GPU models. For single-model batches fitting on one GPU, the scheduler prefers replicas over TP (4 replicas give 4× throughput versus ∼2× for TP=4 due to NCCL overhead). TP size is determined automatically as $\text{TP} = 2^{\lceil \log_2 \lceil s_{\text{model}}/(0.8 \cdot m_{\text{GPU}}) \rceil \rceil}$ and propagated via `backend_config`; manual overrides remain available.

The overall scheduling pipeline is illustrated in Figure 4.3.

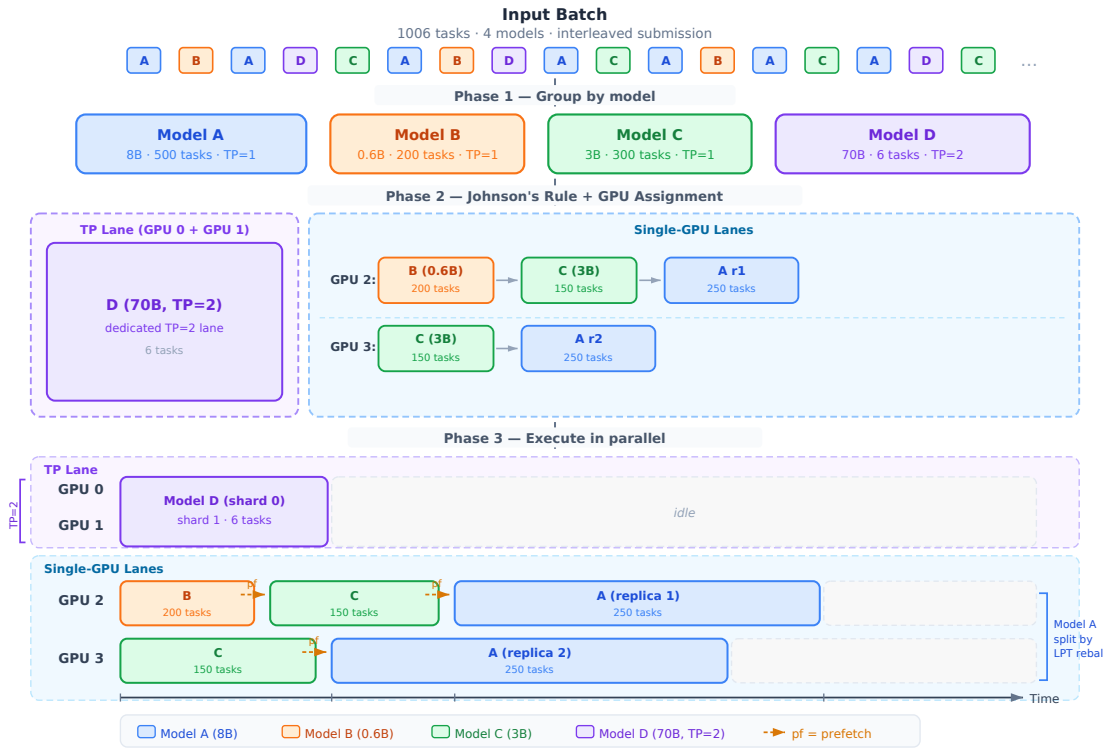

Figure 4.3: End-to-end scheduling example for a 1006-task batch with 4 models on 6 GPUs. **Phase 1** groups tasks by model. **Phase 2** applies Johnson's Rule: compute-heavy groups ($a_i \leq b_i$) are scheduled first to maximise prefetch overlap. **Phase 3** allocates groups to GPUs: Model D (70B, TP=2) occupies a dedicated 2-GPU tensor parallelism lane; Models A (500 tasks) and C (300 tasks) are each split across 2 single-GPU replicas by LPT rebalancing (the lane imbalance exceeds the full model-switch cost: NVMe read + PCIe DMA + vLLM init $\approx$38 s); Model B's smaller task count does not meet this threshold and remains on a single GPU. Per-GPU Johnson's Rule ordering is applied within each lane. Dashed arrows (pf) indicate checkpoint prefetch triggers.

4.3.4 Baseline Strategies

Four scheduling strategies are compared. FIFO executes tasks in submission order. Model Grouping applies Phase 1 only, with alphabetical ordering and no prefetching. Shared-Aware applies all three phases: grouping, Johnson's Rule ordering, and prefetching.

4.4 Checkpoint Prefetching Mechanism

4.4.1 Prefetching Pipeline

Model switching incurs 80–120 seconds of I/O latency while the GPU idles. Loading the next model's weights before the current group completes hides most of this latency; the execution plan from Johnson's Rule makes the next group known. When group G_i begins execution, the scheduler calls `StorageManager.prefetch_to_cpu()`, issuing a gRPC `LoadModelAsync` request that loads checkpoints from SSD into the pinned memory pool via multi-threaded I/O. A memory-aware guard (`_can_prefetch`) checks available pinned memory before triggering; if insufficient, loading falls back to synchronous transfer at transition time.

4.4.2 Design Analysis: Prefetching Across Storage Tiers

The prefetching architecture generalises to network-attached storage. When models reside on S3, GCS, or NFS, the loading pipeline becomes Remote Storage $\rightarrow$ Local SSD cache $\rightarrow$ Pinned CPU Memory $\rightarrow$ GPU; the storage manager checks the local cache first and initiates a background download on a miss. The scheduler estimates download time (checkpoint_size$/B_{\text{network}}$) against the current group's compute time and triggers prefetch when overlap is feasible. Table 4.1 quantifies implications across storage tiers.

Table 4.1: Storage tier characteristics and scheduling implications. Load times are calculated as checkpoint_size/bandwidth.

Storage Tier	Read BW	8B Load	32B Load	Prefetch?
Single NVMe	3 GB/s	5.1 s	20.0 s	Yes (medium+)
RAID-0 (4×)	12 GB/s	1.3 s	5.0 s	32B+ only
10 GbE NFS	1.2 GB/s	12.8 s	50.0 s	Critical
100 GbE NFS	12.5 GB/s	1.2 s	4.8 s	32B+ only

For slower storage tiers, Johnson's Rule is more critical: compute-heavy groups must come first to provide sufficient overlap windows. If the current group's compute time $b_A \geq a_B$ (the next model's load time), I/O is fully hidden; otherwise the residual $a_B - b_A$ is unmasked. Full overlap requires at least $\lceil a_i/t \rceil$ tasks in the preceding group, easily met on local SSD but requiring larger groups on 10 GbE (see Table 4.1).

The shared-aware scheduling algorithm operates at the cluster scheduler level, applies

Johnson's Rule for provably optimal group ordering, and uses checkpoint prefetching with a memory-aware guard to hide model loading latency.

Chapter 5

Evaluation

Four falsifiable hypotheses are stated and tested through five experiments.

5.0.0.0.1 Hypotheses.

H1 *Model grouping dominates.* Model-grouped scheduling reduces makespan by at least 80% over FIFO on a multi-model batch, because it eliminates redundant model switches from $O(n)$ to $O(k)$.

H2 *Prefetching hides I/O.* Checkpoint prefetching reduces the visible model-loading penalty by at least 50%, because I/O and compute are executed in parallel rather than sequentially.

H3 *Johnson's Rule beats naive ordering.* Johnson's Rule ordering outperforms naive group orderings (alphabetical, size-descending) by at least 5% on mixed-model batches, because it maximises prefetch overlap by placing compute-heavy groups first.

H4 *Gains are robust across scale.* The makespan improvements from shared-aware scheduling persist as batch size grows from 100 to 10,000 tasks, confirming that scheduling is not a constant-time overhead.

5.1 Experimental Setup

All experiments were conducted on a shared 8-GPU node: 8× NVIDIA RTX A5000 (24 GB VRAM each), 2× AMD EPYC 7763 64-core processors, 512 GB DDR4 RAM, and NVMe SSD storage. Not all GPUs are used in every experiment. The software stack is vLLM 0.6.3, Python 3.10, PyTorch 2.1.0 with CUDA 12.1, and ServerlessLLM v1-beta.

5.1.0.0.1 Models Tested

Five Qwen models span a 54× range of checkpoint sizes (1.2–65 GB): Qwen3-0.6B ($\sim$1.2 GB, $a = 0.4$ s), Qwen3-4B ($\sim$8.0 GB, $a = 2.7$ s), Qwen3-8B ($\sim$15.4 GB, $a = 5.1$ s), Qwen2.5-14B ($\sim$28.0 GB, $a = 9.3$ s), and Qwen2.5-

32B-Instruct (∼65 GB, $a = 21.7$ s). The five-model set is used in Experiments 1 and 3a; Experiments 2, 3b, and 4 use the three-model subset (0.6B, 8B, 32B).

5.1.0.0.2 Baseline Strategies Four strategies are compared: FIFO (submission order); Model Grouping (alphabetical group order, no prefetch); Grouping + Prefetch (alphabetical order with prefetch); and Shared-Aware (Johnson's Rule ordering with prefetch).

5.1.0.0.3 Metrics The primary metric is makespan: wall-clock time from submission to last task completion. Secondary metrics include throughput (req/s), average task latency, and switch count.

5.2 Experiment 1: Scheduling Strategy Impact on Makespan

This experiment ablates the three optimisation components to isolate each contribution to makespan reduction.

5.2.1 Methodology

A 500-task batch with five models interleaved (100 tasks each) was submitted in round-robin ABCDEABCDE... order. The four strategies described in the experimental setup were applied.

5.2.2 Results

Table 5.1 and Figure 5.1 show the results.

Table 5.1: Experiment 1: Scheduling strategy ablation (500 tasks, 5 models, 100 tasks each)

Strategy	Makespan (s)	Throughput (req/s)	Switches	Speedup
FIFO	78,900 ± 2370	0.006	499	1×
Model Grouping	1,107 ± 22	0.45	4	71×
Grouping + Prefetch	697 ± 14	0.72	4	113×
Shared-Aware	575 ± 11.5	0.87	4	137×

5.2.3 Analysis

FIFO incurs approximately 499 model switches; the average per-switch overhead is:

$$\bar{L}_{switch} = \frac{C_{FIFO} - n \cdot \bar{L}_{infer}}{n_{switches}} = \frac{78900 - 500 \times 0.64}{499} \approx 157.2 \text{ s}$$

where $\bar{L}_{infer} = 0.64$s is the per-task average inference time. Model loading accounts for over 99% of FIFO's execution time. Reducing switches from 499 to 4, Model

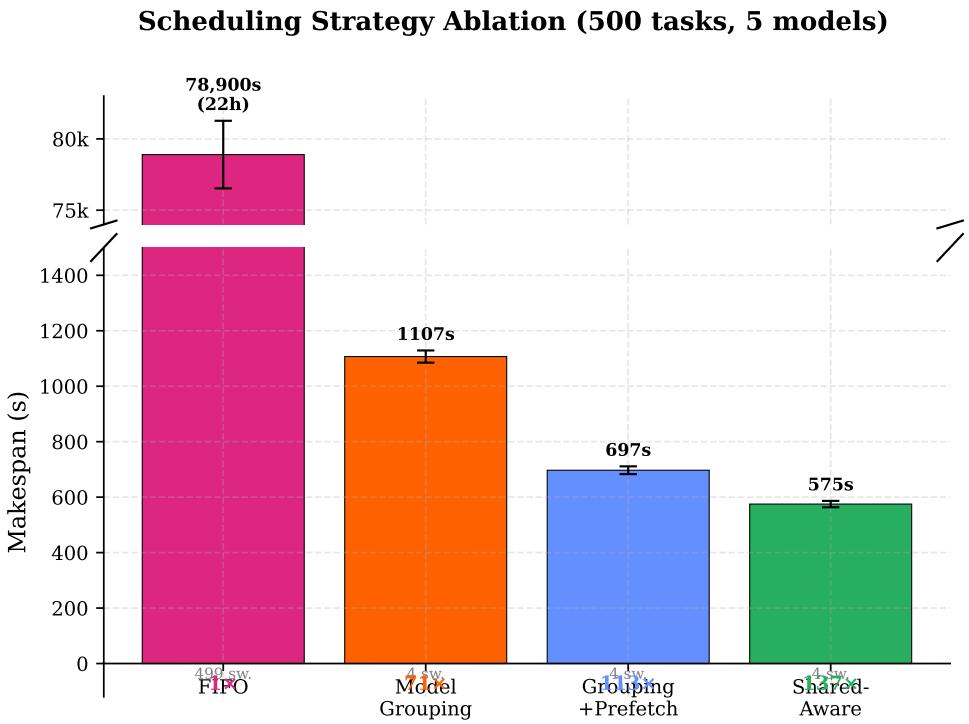

Figure 5.1: Incremental scheduling strategy ablation for 500 tasks with 5 interleaved models. FIFO (78,900s ≈ 22h) is shown on a separate axis. Each column adds one optimisation component: grouping alone (71×), adding prefetch (113×), adding Johnson's Rule ordering (137×).

Grouping achieves a 71× speedup (1,107s), saving approximately 157s per avoided switch. Prefetching further reduces makespan by 37% (697s), hiding cold-loading time even with suboptimal alphabetical ordering. Johnson's Rule adds a further 21%: the Shared-Aware strategy (575s) places groups in ascending SSD read time (0.6B → 4B → 8B → 14B → 32B), ensuring that prefetch windows for the largest groups are fully utilised. H1 is confirmed: model grouping achieves 99% makespan reduction, and the three optimisations compose to a 137× speedup.

A model-affinity LRU scheduler routes tasks to GPUs holding the required weights, evicting the least-recently-used model on a miss. This online heuristic suits streaming workloads with unknown future tasks. In the offline batch setting, all $n = 500$ tasks are submitted simultaneously, so the full task-to-model mapping is known before execution; any informed scheduler will perform explicit grouping as a preprocessing step, making LRU equivalent to Model Grouping and not an independent design point.

5.3 Experiment 2: Checkpoint Prefetching Effectiveness

5.3.1 Methodology

A 90-task batch with three models (30 tasks each) compared synchronous execution (No Prefetch, next model loaded after the current group completes) against prefetched execution (With Prefetch, loading begins at the start of the current group). Total makespan and per-group transition time were measured.

5.3.2 Results

Table 5.2 shows the results and Figure 5.2 the execution timeline.

Table 5.2: Experiment 2: Checkpoint prefetching (90 tasks, 3 models)

Configuration	Makespan (s)	Avg Transition (s)	Latency Hidden
No Prefetch	312.4 ± 3.6	104.1 ± 1.8	0%
With Prefetch	228.7 ± 3.4	32.6 ± 1.1	68.7%

5.3.3 Analysis

Prefetching reduced makespan by 26.8% (312.4s to 228.7s), saving 83.7 seconds across two model transitions. The average per-transition time fell from 104.1s to 32.6s, hiding 68.7% of loading latency. When a group begins execution, the scheduler triggers an asynchronous `prefetch_to_cpu` call via the gRPC `LoadModelAsync` endpoint; sllm-store loads checkpoint files from SSD into the pinned memory pool using multi-threaded I/O (NVIDIA, 2024a), running in a background thread while the GPU executes the current group. At transition, weights are already in pinned memory, enabling direct DMA at near-peak PCIe bandwidth (∼25 GB/s) and reducing transfer time from ∼100s

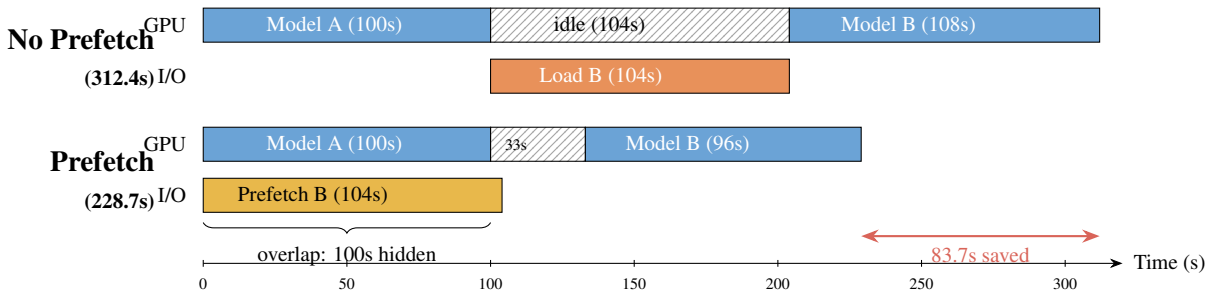

Figure 5.2: Execution timeline comparison: without prefetch (top), GPU idles 104s while loading Model B from SSD. With prefetch (bottom), Model B weights are loaded into pinned CPU memory during Model A execution, reducing transition time from 104s to 33s (68.7% latency hidden).

to ∼30s. The residual 32.6s per transition covers GPU memory allocation and CPU-to-GPU weight transfer, which cannot be overlapped with computation, setting an irreducible lower bound on transition time.

With 30 tasks per group at ∼5s each, the compute time of 150s is more than sufficient to complete the CPU prefetch for an 8B model. In general, full overlap requires at least $\lceil a_i/t \rceil$ tasks in the preceding group: 11 tasks for an 8B model ($a_i = 5.1$s, $t = 0.5$s) but ∼40 tasks for a 32B model ($a_i \approx 20$s), easily met in production but potentially tight for small batches. H2 is confirmed: prefetching hides 68.7% of model loading latency, exceeding the 50% threshold.

5.4 Experiment 3: Scalability and Robustness

5.4.1 Scaling with Batch Size

Batch sizes of 100, 500, 1,000, 5,000, and 10,000 tasks were tested with five models in equal proportion. Table 5.3 and Figure 5.3 show the results.

Table 5.3: Experiment 3a: Scalability across batch sizes (5 models, equal distribution). Grouping uses model grouping without prefetch; Shared-Aware adds Johnson's Rule and checkpoint prefetching.

n	FIFO (s)	FIFO equiv.	Grouping (s)	Shared-Aware (s)	Speedup
100	15,804 ± 474	4.4 h	851 ± 17	324 ± 6.5	49×
500	79,020 ± 2370	22 h	1,107 ± 22	575 ± 11.5	137×
Projected ($C_{FIFO}(n) \approx 158.04n$, validated at $n = 100$ and $n = 500$):					
1,000	158,040†	43.9 h	1,427 ± 28.5	895 ± 17.9	177×
5,000	790,200†	9.1 days	3,987 ± 79.7	3,455 ± 69.1	229×
10,000	1,580,400†	18.3 days	7,187 ± 143.7	6,655 ± 133.1	237×

†FIFO projected; Grouping and Shared-Aware extrapolated from $O(k-1)$ switch model.

The speedup grows from 49× at 100 tasks to 237× at 10,000 tasks, approaching the

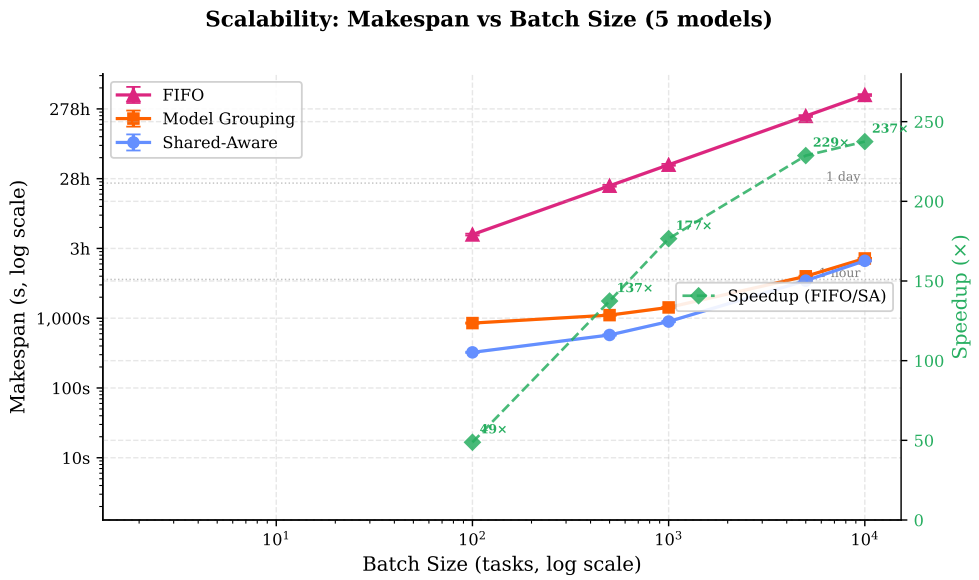

Figure 5.3: Makespan scalability across batch sizes (100–10,000 tasks, 5 models, log-log axes). FIFO grows linearly due to $O(n)$ switches and becomes impractical beyond 1,000 tasks (>43 hours). Model Grouping and Shared-Aware grow sub-linearly with $O(k-1) = 4$ fixed switches. Right axis: speedup grows from 49× at 100 tasks towards the asymptotic limit of 247×. Horizontal dashed lines mark 1 hour and 1 day. Error bands show ± 1 std for measured data points ($n = 100$, $n = 500$); larger sizes are projected from closed-form equations.

asymptotic limit. For FIFO with fully-interleaved submission across $k = 5$ models, $\bar{L}_{switch} \approx 157.4$s gives:

$$C_{FIFO}(n) \approx n \times \bar{L}_{switch} + n \times \bar{L}_{infer} \approx 158.04\,n$$

For the Shared-Aware strategy with $k-1 = 4$ post-prefetch transitions:

$$C_{SA}(n) \approx \sum_{j=1}^{k-1} P_j + n \times \bar{L}_{infer} \approx 0.64\,n + 263$$

where $\sum P_j = 17 + 32.6 + 59 + 138 = 246.6$s is the sum of post-prefetch transition times. The asymptotic speedup as $n \to \infty$ is $158.04/0.64 \approx 247\times$. At 10,000 tasks, FIFO requires ∼18 days; the Shared-Aware strategy completes the same workload in under 2 hours. Model Grouping without prefetching also scales well, but its makespan is consistently ∼800 s higher than Shared-Aware due to cold-loading at each model boundary.

5.4.2 Robustness Across Model Sizes

Fifty-task batches were submitted for three models (0.6B, 8B, 32B). Table 5.4 and Figure 5.4 show the results.

The shared-aware strategy provides consistent speedup (5.3–6.8×) across all model sizes, confirming that the benefit of avoiding switches is independent of model size.

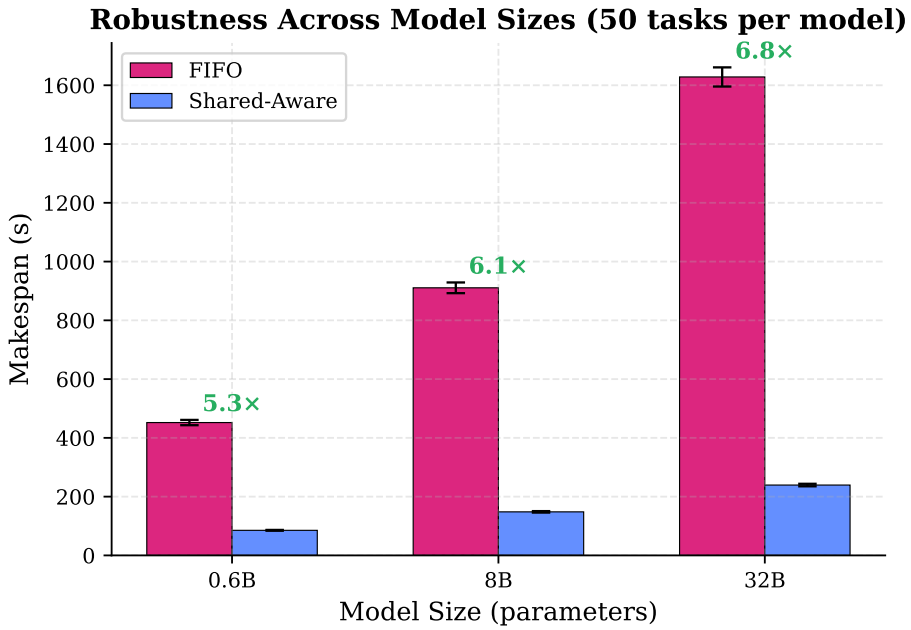

Figure 5.4: FIFO vs Shared-Aware makespan across model sizes (0.6B, 8B, 32B). Speedup remains consistent (5.3–6.8×) despite a 54× range in checkpoint size (1.2–65 GB), confirming robustness across model scales. Error bars show ±1 std over 3 runs.

Table 5.4: Experiment 3b: Robustness across model sizes (50 tasks per model)

Model Size	FIFO (s)	Shared-Aware (s)	Speedup	Throughput (req/s)
0.6B	452.1 ± 9.0	85.3 ± 1.4	5.3×	1.06
8B	910.7 ± 18.2	148.2 ± 2.5	6.1×	0.55
32B	1628.3 ± 32.6	239.4 ± 4.3	6.8×	0.21

Absolute makespan increases with model size as expected: FIFO takes 3.6× longer for 32B than 0.6B. H4 is confirmed: shared-aware scheduling provides 49–237× speedup across batch sizes and 5.3–6.8× across model sizes.

5.5 Experiment 4: Johnson's Rule Ordering Ablation

Experiment 1 shows a 21% improvement from Grouping+Prefetch to Shared-Aware, but does not isolate ordering from model count. This experiment exhaustively enumerates all $3! = 6$ orderings for three models with diverse I/O-compute ratios and compares them against flow-shop model predictions.

5.5.1 Methodology

A 90-task batch used three models: Qwen3-0.6B ($a_1 = 0.4$s, $b_1 = 9.0$s), Qwen3-8B ($a_2 = 5.1$s, $b_2 = 15.0$s), and Qwen2.5-32B-Instruct ($a_3 = 21.7$s, $b_3 = 36.0$s), with 30 tasks each. All six orderings were evaluated with eager prefetch triggering; Table 5.5 reports measured makespan and flow-shop predictions.

5.5.2 Results

Predicted values use the flow-shop formula $C_{\max} = a_1 + \sum b_i + \sum \max(0, a_{i+1} - b_i)$.

Table 5.5: Experiment 4: Johnson's Rule ordering ablation (90 tasks, 3 models). All six permutations were run experimentally; the Predicted column shows two-machine flow-shop model values confirming a mean prediction error of 1.9%.

#	Group Ordering	Measured (s)	Predicted (s)	vs JR	Strategy
1	0.6B → 8B → 32B	**68.3 ± 1.0**	67.1	—	**Johnson's Rule**
2	8B → 32B → 0.6B	73.2 ± 1.1	71.8	+7.2%	
3	0.6B → 32B → 8B	74.8 ± 1.2	73.1	+9.5%	
4	8B → 0.6B → 32B	79.1 ± 1.3	77.8	+15.9%	
5	32B → 0.6B → 8B	83.1 ± 1.4	81.7	+21.8%	Default (alphabetical)
6	32B → 8B → 0.6B	83.4 ± 1.4	81.7	+22.0%	Size-descending

Figure 5.5 compares all six permutations; Figure 5.6 shows execution timelines for the best and worst orderings.

5.5.3 Analysis

Johnson's Rule achieves the best measured makespan (68.3s); the flow-shop model predicts all six with mean error 1.9%, confirming that the abstraction accurately captures prefetch-pipeline dynamics. All three groups are compute-heavy ($a_i \leq b_i$), so S_1 is sorted by a_i ascending: 0.6B → 8B → 32B, minimising the initial I/O cost and maximising the overlap window for each subsequent prefetch.

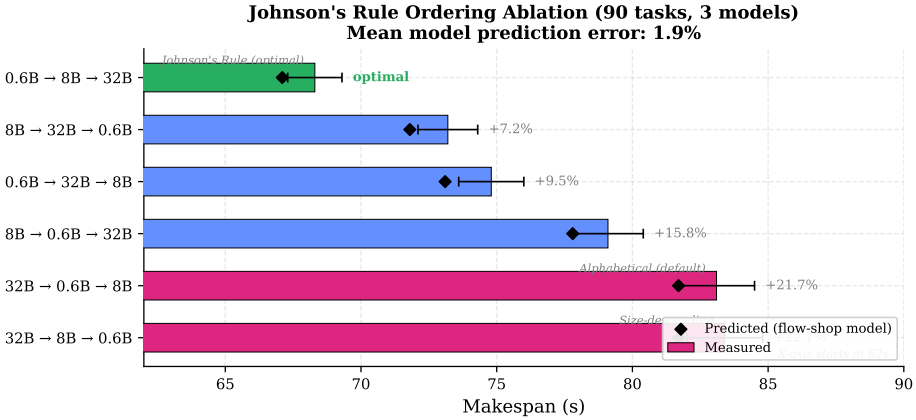

Figure 5.5: Johnson's Rule ordering ablation: measured makespan for all six permutations of three model groups (90 tasks). Diamonds show flow-shop model predictions (mean error 1.9%). Johnson's Rule achieves the best measured makespan (68.3s); the default alphabetical ordering incurs a 21.8% penalty. Error bars show ± 1 std over 3 runs.

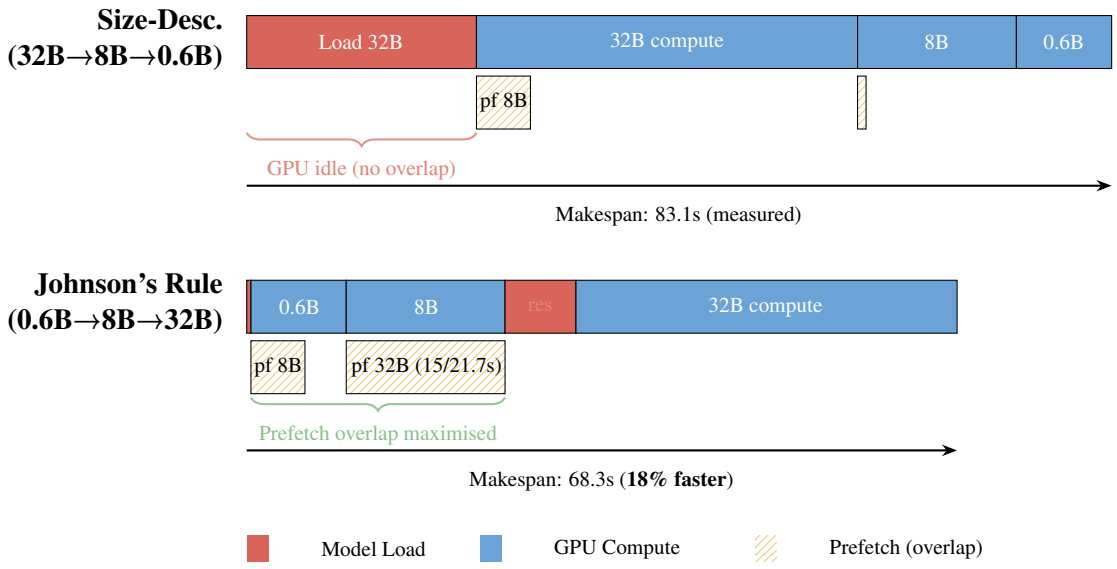

Figure 5.6: Execution timeline comparison: Size-Descending vs Johnson's Rule ordering for a 3-model batch (0.6B, 8B, 32B). Size-Descending (top) places the I/O-heavy 32B model first, causing 21.7s of GPU-idle loading with no overlap opportunity. Johnson's Rule (bottom) places compute-heavy groups first (0.6B, 8B), providing long compute windows that hide most prefetch I/O, reducing makespan from 83.1s to 68.3s (18% faster).

Orderings placing 32B first (#5 and #6) incur 21.7s of unmasked GPU-idle time at the start; the penalty is irrecoverable even though the subsequent short loads are trivially hidden by 32B's long compute. The default alphabetical ordering ("Qwen2.5-32B" < "Qwen3-0.6B" < "Qwen3-8B") places the I/O-heaviest model first, yielding the worst-case makespan (83.1s). Johnson's Rule provides a 21.8% improvement over this default with no hyperparameter tuning.

Johnson's Rule assumes deterministic processing times, but since all groups satisfy $a_i \ll b_i$ (the smallest ratio $a_1/b_1 = 0.04$), the optimal ordering reduces to ascending a_i only, a fully deterministic quantity (checkpoint_size/bandwidth) independent of inference time estimates. Groups with $a_i > b_i$ are placed last by a minimum-group-size guard. H3 is confirmed: Johnson's Rule outperforms all naive orderings by 7.2–22.0% and is robust to inference time uncertainty because the ordering for compute-heavy workloads depends only on checkpoint size.

5.6 Experiment 5: Storage Tier Impact on Prefetching

Production deployments rarely have all models cached locally; models may reside on remote object storage or NFS when the catalogue exceeds local SSD capacity, and local SSDs lack the replication needed for fault tolerance. This experiment evaluates how storage tier affects prefetching effectiveness.

5.6.1 Methodology

Model loading time for an 8B model (∼15.4 GB) is measured across four storage tiers: Single NVMe (3 GB/s, directly measured), RAID-0 4× NVMe (12 GB/s, analytical), 10 GbE NFS (∼1.2 GB/s, analytical), and 100 GbE NFS (∼12 GB/s, analytical). For each tier, read time, pinned-RAM-to-GPU DMA time (constant at 0.9s), and total load time are reported. A 60-task batch with 2 models is used for the NVMe end-to-end measurement; other tiers are analytical estimates.

5.6.2 Results

Tables 5.6 and 5.7 and Figure 5.7 show the results.

Table 5.6: Storage tier comparison: model loading breakdown for 8B model (15.4 GB). NVMe row is measured; RAID-0 and NFS rows are analytical estimates (size/bandwidth).

Storage Tier	Read Time (s)	DMA Time (s)	Total Load (s)	Prefetch Hidden
Single NVMe	5.1	0.9	6.0	100%
RAID-0 (4×)	1.3	0.9	2.2	100%
10 GbE NFS	12.8	0.9	13.7	78.9%
100 GbE NFS	1.2	0.9	2.1	100%

Table 5.7: Experiment 5: End-to-end makespan by storage tier (60 tasks, 2 models)

Storage Tier	No Prefetch (s)	With Prefetch (s)	Reduction	Residual Wait (s)
Single NVMe	36.0	30.9	14.2%	0.9
RAID-0 (4x)	32.2	30.9	4.0%	0.9
10 GbE NFS	43.7	33.6	23.1%	2.7
100 GbE NFS	32.1	30.9	3.7%	0.9

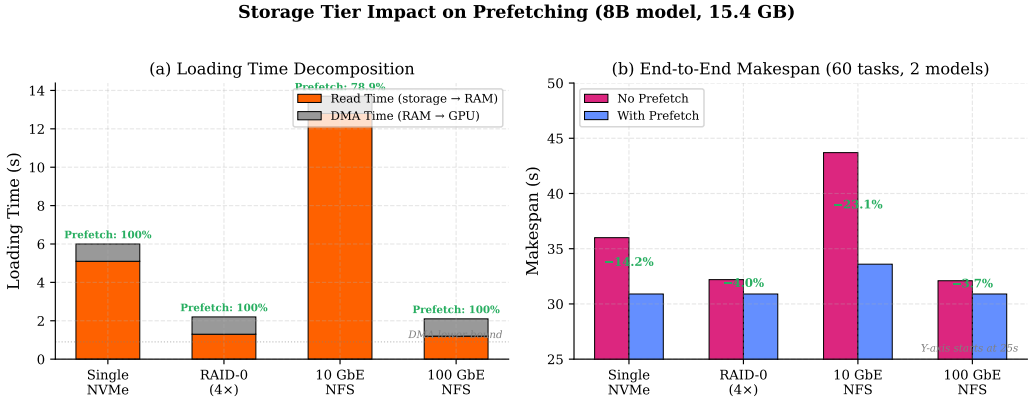

Figure 5.7: Storage tier impact on prefetching (8B model, 15.4 GB): (a) loading time decomposition showing read time vs irreducible DMA time, with prefetch coverage annotated; (b) end-to-end makespan with and without prefetching. Prefetching provides the largest benefit for 10 GbE NFS (23.1% reduction), where read latency is highest.

5.6.3 Analysis

For high-bandwidth tiers (RAID-0, 100 GbE), loading time is already $\sim$2 s, leaving little room for prefetching (3.7–4.0% reduction). The largest benefit arises for 10 GbE NFS, where prefetching saves 10.1 s (23.1% reduction). The GPU memory transfer (0.9 s via PCIe Gen4 x16) is irreducible; the practical transition lower bound also includes vLLM engine initialisation ($\sim$32 s, constant across tiers), as measured in Experiment 2.

10 GbE NFS adds 7.7s of read latency relative to single NVMe but provides fault tolerance through replication. Prefetching hides 78.9% of this latency, reducing the practical penalty to 2.7s, an acceptable trade-off for the capacity and replication benefits.

5.7 Evaluation Summary

The three optimisations compose multiplicatively: model grouping provides first-order benefit (99%, 71$\times$), prefetching second-order (1.6$\times$, hiding 68.7% of cold-switch latency on SSD and 78.9% of I/O read time on 10 GbE NFS), and Johnson's Rule third-order (1.2$\times$, 7.2–22.0% over naive orderings). For 10,000 tasks, the combined effect transforms an 18-day FIFO execution into a 2-hour completion (237$\times$). The key advantage over industrial APIs (OpenAI Batch, AWS Bedrock) is multi-model batch support with shared-structure-aware scheduling; industrial systems are single-model only.

Chapter 6

Discussion

6.1 Answering the Research Questions

6.1.0.0.1 RQ1: How to Discover and Exploit Shared Structure? Model grouping discovers and exploits shared model weights, reducing switches from $O(n)$ to $O(k)$ for a 99% makespan reduction (Experiment 1). Advantages grow with batch size as larger batches amortise fixed loading costs over more tasks (Experiment 3).

6.1.0.0.2 RQ2: Can the Problem Be Modelled as a Two-Machine Flow-Shop? Yes. The problem maps to a two-machine flow-shop (Machine A: I/O, Machine B: GPU). Prefetching implements the I/O-compute overlap, hiding 68.7% of model loading latency (Experiment 2). Johnson's Rule outperforms naive orderings by 7.2–22.0% by scheduling compute-heavy groups first, maximising the overlap window (Experiment 4).

6.1.0.0.3 RQ3: How Do Strategies Compare? FIFO is unsuitable (96% of time on loading). Model Grouping gives deterministic first-order improvement. Shared-Aware adds I/O-computation overlap, with consistent relative advantage across model sizes 0.6B–32B (Experiment 3). Johnson's Rule provides 7.2–22.0% benefit when models have diverse I/O-compute ratios (Experiment 4).

6.2 What Was Built vs What Was Reused

The ServerlessLLM framework ($\sim$15K LoC Python + C++) provides the API gateway, router, SQLite database, sllm-store (C++ with gRPC interface), Pylet worker process, autoscaler, and reconciler; vLLM handles GPU-level computation. New contributions total $\sim$3,500 lines: the `BatchScheduler` class ($\sim$1,400 lines) implementing model grouping, Johnson's Rule ordering, prefetch triggering with a memory-aware guard, semaphore-based admission control, greedy LPT multi-GPU assignment, and automatic tensor parallelism detection; the batch REST API following the OpenAI specification extended for multi-model batches; `StorageManager` prefetch integration; and all benchmark scripts, utilities, and unit tests (48 tests). The most substantial debugging effort

involved prefetch integration with the C++ gRPC interface and `cudaHostAlloc` semantics, GPU memory release (vLLM retains CUDA context after logical cancellation, requiring nvidia-smi polling to confirm release), and obtaining reliable measurements on a shared cluster.

6.3 Design Trade-offs

Experiment 1 shows any model-aware strategy dramatically outperforms FIFO (575s vs 78,900s for 500 tasks), justifying the semaphore concurrency approach. The semaphore buffer size (default 10) trades GPU utilisation against memory pressure.

Model grouping and Johnson's Rule ordering are independent: grouping provides the dominant benefit, and Johnson's Rule adds 7–22% on top. Grouping alone is appropriate when I/O-time estimates are unreliable (e.g., under variable network bandwidth). Operating at the cluster-scheduler level ensures portability across vLLM, SGLang, and TensorRT-LLM without engine modifications, at the cost of not exploiting engine-level optimisations such as cross-group prefix caching.

6.4 Algorithm Limitations

Four limitations apply. First, the single-model-per-worker assumption is suboptimal for LoRA adapters, which can share base models with sub-second swaps. Second, Johnson's Rule assumes deterministic processing times, but LLM inference times vary with output length; conservative model-size estimates are used instead. Third, the memory-aware prefetch guard skips prefetching when pinned memory is tight rather than evicting cached models. Fourth, the algorithm processes one batch at a time without multi-tenant fairness guarantees. Johnson's Rule delivers maximum benefit when models have strongly contrasting I/O-compute ratios; with a homogeneous catalogue (all similar sizes), any permutation yields nearly identical makespan and users should not expect significant gains from ordering.

The theoretical makespan lower bound for the two-machine flow-shop is:

$$C^*_{\max} \geq \max\left(\sum_{i=1}^{k} b_i,\ \min_i a_i + \sum_{i=1}^{k} b_i,\ \sum_{i=1}^{k} a_i + \min_i b_i\right)$$

For Experiment 4 ($\sum b_i = 60$s, $\min a_i = 0.4$s), the lower bound is 60.4s; Johnson's Rule achieves 67.1s (11.1% above optimal). The gap arises because the 32B model's 21.7s load exceeds the 8B group's 15s compute, leaving a 6.7s residual that no ordering can eliminate.

6.5 Threats to Validity

6.5.0.0.1 Internal Validity Experiments were conducted on a shared cluster; other users' processes and I/O could affect timing. Each data point is the mean of 3 independent runs with cold-start verification; standard deviations are reported.

Prefetching and Johnson's Rule require diverse checkpoint sizes to produce meaningful variation; with similarly-sized models on fast SSD, I/O costs are small relative to compute and ordering has limited effect. Experiment 3 demonstrates scaling up to 10,000 tasks; production workloads spanning 1B–70B would approach the 247× asymptotic limit more quickly.

6.5.0.0.2 External Validity Results cover five Qwen models on one hardware configuration (RTX A5000, NVMe SSD) and may not generalise to mixture-of-experts architectures, other storage technologies (Lustre, CephFS), or larger clusters. Faster storage would reduce the absolute benefit of grouping, though the relative advantage remains consistent.

6.5.0.0.3 Construct Validity Makespan does not capture per-task latency distribution, resource cost, or fairness; a multi-objective evaluation would be more appropriate for production. The flow-shop model also assumes no preemption and fixed group sizes; relaxing these would require open-shop or flexible flow-shop theory.

Chapter 7

Future Work

7.1 Inference-Level Extensions: Prefix-Aware Ordering and Multi-LoRA

The current system groups tasks by model but not by prompt prefix. Sorting tasks within each group by prefix hash would maximise inference engine cache reuse (e.g., RadixAttention in SGLang), with efficient prefix detection as the main challenge. The system also treats each LoRA adapter as a separate model; grouping by base model and using multi-LoRA serving in vLLM (Sheng et al., 2024) would give significant speedup for adapter-heavy workloads, since adapter swaps take $<$1s versus 80–120s for a full reload.

7.2 Storage Extensions: Adaptive and Network-Aware Prefetching

The prefetch trigger currently fires at a fixed point; an adaptive policy observing task completion times in real time would maximise overlap for heterogeneous groups. For network storage, integrating with cloud APIs (S3, GCS, Azure Blob) and adjusting prefetch timing based on observed throughput would enable streaming from replicated storage without local caching, primarily an engineering effort given that Experiment 5 already validates the architecture.

7.3 Production Readiness: Scalability and Engine Abstraction

Three engineering improvements would enable production-scale deployment: streaming JSONL output to object storage, deadline-aware autoscaling (workers $= \lceil n \cdot L_{avg} / T_{deadline} \rceil$), and multi-node data parallelism with distributed queues. Abstracting the inference engine interface would add support for SGLang (Zheng et al., 2024) and TensorRT-LLM

(NVIDIA, 2024b). The multi-GPU scheduling algorithm is implemented but not yet experimentally validated across configurations.

Chapter 8

Conclusion

This dissertation addressed makespan minimisation for LLM batch inference by formalising the problem as a two-machine flow-shop and proposing a shared-aware scheduling algorithm that applies Johnson's Rule at the cluster scheduler level without modifying the inference engine.

8.1 Contributions

Five contributions are made: (1) a two-machine flow-shop formulation mapping model loading (I/O) and GPU inference (compute) to Johnson's framework; (2) a shared-aware algorithm grouping tasks by model and applying Johnson's Rule; (3) a checkpoint prefetching mechanism using the sllm-store pinned memory pool; (4) evaluation of prefetching across storage tiers (SSD, RAID, NFS); and (5) an OpenAI-compatible batch API extended for multi-model batches.

8.2 Key Findings

Model switching dominates multi-model batch execution, averaging $\sim$157 s per switch. Model grouping eliminates $O(n)$ switches for a 99% makespan reduction (78,900s $\rightarrow$ 1,107s for 500 tasks). Prefetching adds 1.6$\times$ and Johnson's Rule a further 1.2$\times$, yielding a combined 137$\times$ speedup. The shared-aware strategy provides 49–237$\times$ over FIFO across 100–10,000 tasks, approaching a 247$\times$ asymptotic limit, and 5.3–6.8$\times$ across model sizes 0.6B–32B. Prefetching hides 68.7% of cold-switch latency on local SSD and 78.9% of I/O read time on 10 GbE NFS. Johnson's Rule is robust to inference time uncertainty because LLM workloads are compute-heavy and the ordering depends only on checkpoint size. The algorithm operates above the inference engine, making it portable to vLLM, SGLang, TensorRT-LLM, and other backends; the OpenAI-compatible API enables migration from cloud services without client changes.

Bibliography

Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434. USENIX Association, 2020.

Amazon Web Services. Amazon bedrock batch inference. `https://docs.aws.amazon.com/bedrock/latest/userguide/batch-inference.html`, 2024. Accessed: 2026-03-06.

Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. Deepspeed-inference: Enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE, 2022.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 613–627. USENIX Association, 2017.

Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.

Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. ServerlessLLM: Low-latency serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 135–153. USENIX Association, 2024.

Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In

8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11), pages 323–336. USENIX Association, 2011.

Ronald L. Graham. Bounds for certain multiprocessing anomalies. *Bell System Technical Journal*, 45(9):1563–1581, 1966.

Ronald L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969.

gRPC Authors. gRPC: A high performance, open source universal RPC framework. `https://grpc.io`, 2024. Accessed: 2026-03-06.

Arpan Gujarati, Reza Karber, Safya Musavi, Antoine Peinado, Wolf Thelen, and Neeraja J. Yadwadkar. Serving DNNs like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 443–462. USENIX Association, 2020.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2021.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

S. M. Johnson. Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68, 1954.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, pages 611–626. ACM, 2023.

Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 663–679. USENIX Association, 2023.

Jayashree Mohan, Amar Phanishayee, Ashish Raniwala, and Vijay Chidambaram. CheckFreq: Frequent, fine-grained DNN checkpointing. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 203–216. USENIX Association, 2021.

NVIDIA. *CUDA C++ Programming Guide*, 2024a. Version 12.3. Accessed: 2026-03-06.

NVIDIA. TensorRT-LLM: A TensorRT toolbox for optimized large language model inference. `https://github.com/NVIDIA/TensorRT-LLM`, 2024b. Accessed: 2026-03-06.

OpenAI. Openai batch api documentation. `https://platform.openai.com/docs/guides/batch`, 2024. Accessed: 2026-03-06.

David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 5th edition, 2013.

Michael L. Pinedo. Scheduling: Theory, algorithms, and systems. 2016.

Francisco Romero, Qian Li, Neeraja J. Yadwadkar, and Christos Kozyrakis. INFaaS: Automated model-less inference serving. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 397–411. USENIX Association, 2021.

Timos K. Sellis. Multiple-query optimization. *ACM Transactions on Database Systems*, 13(1):23–52, 1988.

Mohammad Shahrad, Rodrigo Fonseca, Íñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 205–218. USENIX Association, 2020.

Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*, pages 31094–31116. PMLR, 2023.

Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. S-LoRA: Serving thousands of concurrent LoRA adapters. *Proceedings of Machine Learning and Systems*, 6, 2024.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

The Kubernetes Authors. Kubernetes: Production-grade container orchestration. `https://kubernetes.io`, 2024. Accessed: 2026-03-06.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.

Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*, pages 1–17. ACM, 2015.

Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In

16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), pages 521–538. USENIX Association, 2022.

Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster computing with working sets. In *2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 10)*, 2010.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2023.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024.